

VoteChain

A Decentralised Voting System Using Ethereum Smart Contracts

Yash Yadav

Department of Physics, Indian Institute of Technology Kanpur

2024–25

Abstract. Traditional electronic voting systems suffer from single points of failure, opaque audit trails, and centralised control that requires trust in an authority. This project presents *VoteChain*, a full-stack decentralised voting platform built on the Ethereum blockchain. The system enforces election integrity through immutable smart-contract logic written in Solidity, role-based access control separating super-admin, election-admin, and voter roles, and an event-sourced frontend that reads on-chain data gas-free. Two voting modes are supported: a transparent one-vote-per-wallet ballot and a privacy-preserving *commit–reveal* scheme implemented fully on-chain, in which votes are submitted as cryptographic commitments and tallied only after a separate reveal phase. Administrative setup is batched so that an entire candidate slate or voter roll is registered in a single transaction. The contract is deployed on the Ethereum Sepolia testnet and is validated by a 48-case Hardhat test suite covering access control, the election lifecycle, double-vote prevention, and the commit–reveal flow. The report also documents the security engineering applied to the repository itself, including secret-management remediation.

Live contract:

<https://sepolia.etherscan.io/address/0xBD29A19DBD25bD4d97C09b65862CB7061A9b3011>

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Scope and Limitations	2
2	Background	2
2.1	Ethereum and Smart Contracts	2
2.2	Solidity	2
2.3	MetaMask and Ethers.js	2
3	System Architecture	3
3.1	Overview	3
3.2	Component Summary	3
4	Smart Contract Design	3
4.1	State and Data Structures	3
4.2	Election Lifecycle	3
4.3	Access Control	3
4.4	Vote Integrity (Plain Mode)	4
4.5	Commit–Reveal Privacy (On-Chain)	4
4.6	Batched Administration	4
4.7	Event Architecture	4
5	Frontend Implementation	5
5.1	Architecture Decisions	5
5.2	Identity and the “Login” Model	5
5.3	Event-Querying Strategy	5
5.4	Commit–Reveal in the UI	5
6	Security Analysis	5
6.1	Threat Model	5
6.2	Repository Security Engineering	5
6.3	Known Limitations	6
6.4	Testing and Verification	6
6.5	Gas Analysis	6
7	Deployment	6
7.1	Technology Stack	6
7.2	Deployment Process	7
8	Results and Demo Flow	7
9	Discussion	7
10	Future Work	7
11	Conclusion	8

1. Introduction

Elections are foundational to democratic systems, yet existing electronic voting infrastructure frequently concentrates control in a single authority, making comprehensive external audit difficult. Paper-based systems are slow and prone to physical manipulation. Blockchain-based voting addresses the integrity and auditability dimensions of this problem by recording votes as cryptographically signed on-chain transactions that are visible to all participants and modifiable by none.

1.1 Motivation

Recent public discourse around election integrity motivates a system in which:

- Every vote is an immutable on-chain transaction.
- Counting is deterministic and publicly verifiable.
- No central authority controls data after submission.
- The complete audit trail is available to any observer.

1.2 Scope and Limitations

Blockchain guarantees *integrity* (votes cannot be modified) and *transparency* (totals are publicly verifiable). It does not, by itself, guarantee voter *identity* (a wallet is not a verified person). Vote *privacy* on a public chain is addressed in this project by an on-chain commit–reveal protocol. Identity verification is delegated to an off-chain admin whitelist: eligible voters register their public wallet address with the election authority, which authorises those addresses on-chain. The authentication primitive is therefore wallet ownership; a voter proves control of a registered address by signing with its private key; nothing is ever typed as a password.

2. Background

2.1 Ethereum and Smart Contracts

Ethereum is a decentralised, Turing-complete blockchain on which programs called *smart contracts* execute deterministically across all network nodes. A contract deployed to an address is immutable; its logic cannot be changed after deployment. Transactions that call contract functions are cryptographically signed by the sender’s private key, creating a tamper-evident audit trail. Gas fees denominated in Ether compensate validators for computation.

2.2 Solidity

Solidity is the primary high-level language for Ethereum smart contracts. Features used in this project include *mappings* (hash-map storage), *enums* (the election state machine), *events* (cheap on-chain logging queried by the frontend), *modifiers* (access-control predicates), and *keccak256* hashing (the commit–reveal scheme).

2.3 MetaMask and Ethers.js

MetaMask is a browser extension that manages Ethereum key pairs and signs transactions on behalf of the user. Ethers.js v6 is a JavaScript library that provides a high-level API for connecting to nodes, instantiating contracts, and querying event logs without gas cost.

3. System Architecture

3.1 Overview

All persistent state lives on-chain. The frontend is a pure read/write client with no backend server. Read operations (fetching election lists, candidates, voter rolls, vote counts) query the contract's event logs; a gas-free operation using the Ethereum `eth_getLogs` JSON-RPC call. Write operations (creating elections, casting votes, etc.) send signed transactions via MetaMask.

Voter/Admin Browser (React + Ethers.js + MetaMask) → Ethereum RPC (Alchemy / public Sepolia) → Deployed `VotingSystem` contract (Sepolia)

3.2 Component Summary

Component	Technology	Role
Smart contract	Solidity 0.8.20	On-chain state, access control, event log
Deploy script	Hardhat + Ethers	Sepolia deployment
Test suite	Hardhat + Chai (48 tests)	Correctness and security verification
Frontend	React + Vite	Admin and voter UI, wallet integration
Wallet	MetaMask	Key management, transaction signing
RPC provider	Alchemy / Sepolia	Ethereum node access

4. Smart Contract Design

4.1 State and Data Structures

Elections are stored in a `private` mapping to prevent direct external enumeration; the frontend reconstructs the list from emitted events. Each `Election` contains nested mappings for candidates, voters, and admin addresses, providing $\mathcal{O}(1)$ lookup. A per-election boolean `commitReveal` selects the voting mode.

```

1  enum ElectionState { Created, Active, Reveal, Ended }
2
3  struct Voter {
4      bool isAuthorized;
5      bool hasVoted;           // plain: voted | commit-reveal: revealed
6      uint votedCandidateId;
7      bytes32 commitment;     // keccak256(candidateId, secret, voter)
8      bool hasCommitted;
9  }

```

4.2 Election Lifecycle

Plain elections progress `Created` → `Active` → `Ended`. Commit-reveal elections insert a reveal window: `Created` → `Active` (commit) → `Reveal` → `Ended`. Candidates may only be added in `Created`; results may only be read in `Ended`. The state machine prevents premature result leakage and invalid transitions.

4.3 Access Control

A two-tier role model is enforced by `require` guards:

- **Super Admin** (contract deployer): creates elections, assigns election-level admins.

- **Election Admin:** adds candidates, authorises voters, starts/ends the election, opens the reveal phase.

These modifiers are the authoritative enforcement layer: even if the user interface were bypassed, the contract itself rejects administrative transactions from non-admin wallets.

4.4 Vote Integrity (Plain Mode)

Four on-chain checks prevent vote manipulation: the election must be **Active**; the voter must be authorised; the voter must not have already voted (the `hasVoted` flag is set atomically within the same transaction, making double-spending impossible); and the candidate id must be in range.

4.5 Commit–Reveal Privacy (On-Chain)

The privacy scheme is implemented as two phases.

1. **Commit:** during the **Active** window the voter submits $H = \text{keccak256}(\text{candidateId} \parallel \text{secret} \parallel \text{voterAddr})$. The hash reveals nothing about the choice, and binding the voter's address prevents a commitment from being copied and replayed by another wallet.
2. **Reveal:** after the admin opens the **Reveal** window, the voter submits the plaintext (`candidateId`, `secret`). The contract recomputes H and counts the vote only if it matches the stored commitment. Unrevealed commitments are never tallied.

```

1 function revealVote(uint _electionId, uint _candidateId, string memory
  _secret) public {
2     // ... state and authorization checks ...
3     bytes32 check = keccak256(abi.encodePacked(_candidateId, _secret, msg.
      sender));
4     require(check == voter.commitment, "Reveal does not match commitment");
5     voter.hasVoted = true;
6     e.candidates[_candidateId].voteCount++;
7     emit VoteRevealed(_electionId, msg.sender);
8 }

```

4.6 Batched Administration

To minimise the number of wallet signatures during setup, the contract exposes `addCandidates(id, string[])` and `authorizeVoters(id, address[])`, which register an entire list in a single transaction. A full election setup therefore requires three signatures (create, add all candidates, authorise all voters) rather than one per candidate and per voter. This is a deliberate design choice: every state change must be signed — there is no mechanism to skip confirmation for individual writes — so the correct optimisation is to reduce the *number* of transactions, not to weaken the signing requirement.

4.7 Event Architecture

Ten events enable gas-free data retrieval: `ElectionCreated`, `CandidateAdded`, `VoterAuthorized`, `VoteCast`, `VoteCommitted`, `VoteRevealed`, `ElectionStarted`, `RevealStarted`, `ElectionEnded`, and `AdminAssigned`. No event parameter is declared `indexed`; the frontend fetches each event type and filters by election id in JavaScript.

5. Frontend Implementation

5.1 Architecture Decisions

The application uses a single `view` state variable (`"connect" | "admin" | "voter"`) set atomically with all auth state, avoiding a routing race condition. All sub-components are defined at module scope so React never remounts them on re-render, which preserves input focus across keystrokes.

5.2 Identity and the “Login” Model

A deliberate design point: *the wallet is the login*. Connecting MetaMask proves the user controls a particular address; there is no address field to type, because typing an address would let anyone impersonate any voter. On connecting, the app reads `superAdmin()` and routes the deployer wallet to the admin dashboard and every other wallet to the voter interface. The voter view displays an explicit **Authorized / Not authorized** badge for the connected wallet, and the admin portal refuses to render for non-admin wallets (defence in depth atop the on-chain modifiers).

5.3 Event-Querying Strategy

Public Sepolia RPC nodes reject `eth_getLogs` queries spanning more than $\sim 500,000$ blocks, so the frontend computes a safe range:

```
1 const latest = await provider.getBlockNumber();
2 const fromBlock = Math.max(0, latest - 500000); // ~70 days
```

5.4 Commit–Reveal in the UI

Creating an election offers a “Private (commit–reveal)” toggle. In a private election the voter first commits a hashed vote (the secret is auto-generated if left blank and saved in `localStorage` for convenience), the admin then opens the reveal phase, and the voter reveals to have the vote counted. The reveal form can pre-fill the saved commitment or accept a manually entered candidate and secret.

6. Security Analysis

6.1 Threat Model

Attack	Protection	Mechanism
Vote tampering	Blockchain immutability	EVM consensus
Double voting	<code>hasVoted</code> flag	Set atomically with the vote
Unauthorised voting	<code>isAuthorized</code> check	Admin whitelist per election
Vote privacy	Commit–reveal	Hashed commitment, delayed reveal
Commitment replay	Address-bound hash	<code>keccak256(..., msg.sender)</code>
Fake result reporting	On-chain counting	Tally incremented by contract
Admin escalation	Modifier guards	<code>onlySuperAdmin/onlyElectionAdmin</code>
Sybil (fake wallets)	Off-chain identity	Admin whitelisting boundary

6.2 Repository Security Engineering

Beyond the contract, the project’s source repository was hardened. An early commit had inadvertently tracked a `.env` file containing a deployer private key and API credentials, along with

the `node_modules` directory. Remediation involved rotating all exposed credentials, rewriting Git history to purge the secrets from every commit, correcting the `.gitignore` patterns (which were root-anchored and failed to match nested directories), and adding a committed `.env.example` template. This mirrors a real incident-response workflow and underlines that key management is part of a voting system’s security boundary.

6.3 Known Limitations

A wallet address is not a verified human identity; the system assumes off-chain identity verification before authorisation. The contract has not undergone a formal third-party audit. A compromised admin key grants election control — a multi-signature admin model is identified as future work. The commit–reveal scheme is vulnerable to weak-secret dictionary attacks if voters choose low-entropy secrets.

6.4 Testing and Verification

Correctness is verified by a Hardhat test suite of **48 passing tests** executed on a local EVM. Coverage includes: deployment and role assignment; election creation and the full state machine; single and batched candidate/voter registration; plain voting with double-vote, authorisation, and candidate-range checks; and the complete commit–reveal flow (commit acceptance and rejection, reveal-phase gating, commitment matching, double-reveal prevention, and mode isolation between plain and private elections). The suite runs with `npm test`.

6.5 Gas Analysis

Operation	Gas (approx.)	Cost @ 10 Gwei
Deploy contract	~1,300,000	~0.013 ETH
createElection	~120,000	~0.0012 ETH
addCandidates (per name)	~75,000	~0.0008 ETH
authorizeVoters (per voter)	~45,000	~0.0005 ETH
vote / commitVote / revealVote	~55,000	~0.0006 ETH
Read operations / event queries	0	Free

All read operations are view-function calls or event-log queries and are gas-free. Only state-mutating transactions incur gas; voters pay only their own `vote/commit/ reveal` gas, while administrative gas is borne by the admin wallet.

7. Deployment

7.1 Technology Stack

Layer	Technology / Version
Smart contract language	Solidity 0.8.20
Contract framework	Hardhat 2.22.x
Test framework	Hardhat + Chai matchers (ethers v5)
Blockchain	Ethereum Sepolia testnet
Frontend framework	React 19 + Vite
Blockchain library	Ethers.js v6 (frontend)
Wallet	MetaMask

7.2 Deployment Process

1. **Environment.** Configure a root `.env` with the Sepolia RPC URL and deployer private key (the Hardhat config resolves this file from the repository root).
2. **Compile.** `npx hardhat compile`.
3. **Test.** `npx hardhat test` (48 passing).
4. **Deploy.** `npx hardhat run scripts/deploy.js --network sepolia`.
5. **Configure.** Update `CONTRACT_ADDRESS` in `contract.js` with the deployed address.
6. **Frontend.** `cd voting-frontend && npm run dev`.

Live contract: <https://sepolia.etherscan.io/address/0xBD29A19DBD25bD4d97C09b65862CB7061A9b3011>

8. Results and Demo Flow

A complete end-to-end demonstration proceeds as follows:

1. Admin connects the deployer wallet and is auto-routed to the admin dashboard.
2. Admin creates an election (optionally toggling private commit-reveal mode).
3. Admin adds the entire candidate slate in one transaction.
4. Admin authorises the full voter roll in one transaction.
5. Admin starts the election; a live indicator appears.
6. Each voter connects *their own* wallet; the authorisation badge confirms eligibility.
7. Voters cast (plain) or commit (private) their votes.
8. For private elections the admin opens the reveal phase and voters reveal.
9. Admin ends the election; results become readable and a bar chart renders with the winner.

All intermediate steps are verifiable on Sepolia Etherscan by inspecting the contract's transaction history.

9. Discussion

The core contribution of this architecture is the **trust model**. In a traditional system, a server administrator can modify vote counts without detection. Here, the counting logic is encoded in contract bytecode verified against the on-chain hash, executed identically by all EVM nodes, and every mutation is permanently logged. An auditor needs only to read the chain, no privileged access to a server is required. The honest engineering trade-offs are acknowledged throughout: blockchain guarantees integrity but not identity, and every on-chain write has a gas cost, creates friction at scale, partially mitigated here by batching and event-sourced reads.

10. Future Work

- **ZK-proof integration.** Zero-knowledge proofs to prove voter authorisation without revealing the chosen candidate, achieving authentication and privacy simultaneously.
- **Indexed events.** Redeploy with indexed election ids to enable efficient on-chain topic filtering.
- **IPFS metadata.** Store candidate photos, manifestos, and descriptions off-chain with on-chain content hashes.
- **Hardware-wallet key management.** Replace software key custody with Ledger/Trezor integration for high-security deployments.
- **Multi-signature admin.** Require k -of- n admin signatures to start or end an election, removing the single-point-of-failure admin key.

- **Real identity layer.** Integrate institutional SSO or government-ID-linked wallet binding for authenticated voter registration.
- **Reveal-window enforcement.** Enforce the stored start/end timestamps on-chain so phases close automatically.

11. Conclusion

VoteChain demonstrates that a functional, secure, and auditable voting system can be built on Ethereum with a compact Solidity contract and a React frontend. The key design decisions immutable on-chain state, event-sourced reads, role-based access control, wallet-based authentication, batched administration, and a genuine on-chain commit reveal privacy scheme collectively produce a system where vote counts cannot be tampered with after submission, double voting is prevented cryptographically, results are publicly verifiable without any trusted intermediary, and the admin can change only the lifecycle of an election, never the votes. Validated by a 48-case test suite and deployed on Sepolia, the project also surfaces honest trade-offs: blockchain secures integrity but not identity or privacy on its own, and the commit reveal extension and proposed ZK and multi-signature work chart a credible path toward real-world deployment.

References

- [1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.
- [2] V. Buterin, “Ethereum: A next-generation smart contract and decentralised application platform,” 2014.
- [3] Solidity Documentation, *Solidity v0.8.20*, Ethereum Foundation, 2024. <https://docs.soliditylang.org>
- [4] R. Rimple, *Ethers.js Documentation v6*, 2024. <https://docs.ethers.org/v6/>
- [5] R. Cramer, R. Gennaro, and B. Schoenmakers, “A secure and optimally efficient multi-authority election scheme,” *EUROCRYPT '97*, Springer LNCS, 1997.
- [6] J. Benaloh and D. Tuinstra, “Receipt-free secret-ballot elections,” *Proc. 26th ACM STOC*, 1994.
- [7] Nomic Foundation, *Hardhat Documentation*, 2024. <https://hardhat.org>